

Medistocks documentation

Medistocks: Project Requirements Document

- **Version: 1.0**
- **Date: September 27, 2025**

Project: Medistocks - A B2B Marketplace for Near-Expiry Pharmaceutical Products

1.1. Project Overview

Medistocks is a digital platform designed to connect pharmaceutical distributors and retailers. The primary goal is to create a marketplace for selling near-expiry pharmacy items at discounted prices, thereby reducing waste and providing cost-effective inventory for retailers. The platform will facilitate product discovery, secure transactions, and communication, and will be enhanced with features like AI-powered demand forecasting and in-app advertising.

1.2. Key Objectives

- To create a centralized marketplace for near-expiry pharmaceutical goods.
- To provide a secure onboarding and verification process for professional users.
- To facilitate efficient product discovery through location-based filtering.
- To enable secure and verifiable transactions between buyers and sellers.
- To offer data-driven insights to help users optimize their stock.

- To create a monetization channel through user subscriptions and advertisements.

Modules:-

1. User Authentication & Onboarding Module

This module will handle the entire user lifecycle from registration to logging in and managing their core profile.

- Features:
 - Mobile number verification via OTP.
 - Role selection (Retailer/Distributor).
 - User registration with detailed information (Name, Address, PAN, Aadhaar, Pharmacy License No.).
 - Secure document upload (Aadhaar, PAN, License) to AWS S3.
 - Subscription package selection (Silver, Gold, Platinum) and payment integration.
 - User profile management (update name, profile image, phone number).
 - Login system using mobile number.
-

2. Admin Panel & Verification Module

This is the central control panel for the administrator to manage the platform, users, and content.

- Features:
 - Dashboard with platform statistics.

- User verification queue.
 - View user-submitted details and documents.
 - Approve or reject user registrations with a reason.
 - Manage advertisement requests (Approve/Reject).
 - Content management system to post educational/promotional videos.
 - Management of subscription packages.
-

3. Product & Inventory Management Module

This core module deals with the creation, display, and management of product listings and personal stock.

- Features:
 - For Distributors/Retailers:
 - Post a new product for sale (Product Image, Type, Quantity, Discount Price, Expiry Date).
 - "My Inventory" feature to add and track their own stock (Purchase Date, Quantity, Expiry).
 - For Retailers:
 - View a feed of all products posted by distributors.
 - Advanced filtering system (e.g., filter by location within 10km, 50km, 100km).
 - Search functionality for products.
-

4. Order & Transaction Management Module 🤝

This module handles the entire workflow from a retailer expressing interest to a completed deal.

- Features:
 - Retailer sends a "Purchase Request" to a distributor for a product.
 - Distributor can accept or reject the purchase request.
 - An in-app chat feature is enabled between the two parties *only after* a request is accepted.
 - Distributor can generate an invoice within the system.
 - A unique verification code is sent to the retailer to confirm the transaction.
-

5. AI-Powered Demand Forecasting Module 🤖

A specialized module to provide business intelligence and sales suggestions to users.

- Features:
 - Analyze data based on weather patterns, seasons, and geographical location.
 - Suggest products that are currently in high demand.
 - Provide insights to both retailers and distributors to optimize their inventory and sales strategy.
-

6. Advertisement Management Module 📢

This module allows users to promote their services or products on the platform.

- Features:
 - For Distributors/Retailers:
 - Create an "Ad Request" by uploading a banner, content, and description.
 - Select the duration for the ad to be displayed.
 - Payment gateway for ad placement.
 - For Admins:
 - Review and approve/reject ad requests.
 - Platform Feature:
 - Display approved ads on user dashboards (primarily for retailers).
-

7. Content & Notifications Module

This module focuses on delivering content from the admin to users and providing crucial system alerts.

- Features:
 - Video feed where retailers and distributors can watch content posted by the admin.
 - A centralized notification system for alerts like:
 - Account approval/rejection.
 - New purchase requests.
 - Chat messages.
 - Invoice generation.

Each module explantion:-

Module 1: User Authentication & Onboarding

Objective: To provide a secure and seamless process for new Retailers and Distributors to register, verify their identity, subscribe to a plan, and log in to the platform.

1. Detailed Workflow & Procedure

The onboarding process is a multi-step flow that ensures all necessary information and verification is completed before a user gains access.

Step 1: Mobile Number Verification (The Entry Point)

- A new user visits the Medistocks app/website and is prompted to enter their 10-digit mobile number.
- The system sends a 6-digit One-Time Password (OTP) to this number via SMS.
- The user enters the OTP on the verification screen.
- System Check: Upon successful OTP verification, the system checks if the mobile number exists in the users table.
 - If number does NOT exist: The user is a new registration and proceeds to Step 2.
 - If number exists: This is a returning user (login attempt). The system checks their approval_status.
 - If approved: Login is successful. The user is issued an access token and redirected to their respective dashboard.
 - If pending or rejected: The user is shown a status screen (e.g., "Your application is still under review" or "Your application was rejected. Reason: [rejection_reason]").

Step 2: Role Selection & Information Entry

- The new user is presented with a form to select their Role (Retailer or Distributor).
- They fill in their personal and professional details:

- Full Name
- Business/Shop Address
- PAN Card Number
- Aadhaar Number
- Pharmacy License Number

Step 3: Document Upload

- The user is prompted to upload scanned copies or clear photos of their documents:
 - PAN Card
 - Aadhaar Card
 - Pharmacy License
- Backend Process: These files are sent to the server, which then securely uploads them to a dedicated AWS S3 bucket. The server gets back a unique URL for each file (e.g., s3.amazonaws.com/medistocks-docs/user_1_pan.pdf). These URLs are what we will store in the database, not the files themselves.

Step 4: Subscription & Payment

- The user is shown the available subscription packages (Silver, Gold, Platinum) with their features and prices.
- After selecting a package, they are redirected to a secure payment gateway (e.g., Razorpay, Stripe) to complete the transaction.
- The payment gateway notifies our backend system about the payment status (success or failure).

Step 5: Application Submission & Pending Status

- Upon successful payment, the user's complete profile is created in the database.

- The approval_status field for this new user is set to pending by default.
- The user is shown a confirmation screen: "Thank you for registering! Your application is now under review by our team. You will be notified once it is approved." The user's journey ends here until the Admin takes action.

2. Database Schema

We'll need three primary tables for this module to function correctly.

users

1. This is the main table to store all user information. | Column Name | Data Type | Constraints & Notes || :--- | :--- | :--- ||
2. id | INT / BIGINT | Primary Key, Auto Increment ||
mobile_number | VARCHAR(15) | UNIQUE, Not Null. Used for login. || name | VARCHAR(255) | Not Null || role | ENUM('retailer', 'distributor') | Not Null || address | TEXT | Nullable || pan_number | VARCHAR(10) | UNIQUE, Not Null || aadhaar_number | VARCHAR(12) | UNIQUE, Not Null || pharmacy_license_number | VARCHAR(50) | UNIQUE, Not Null || pan_document_url | VARCHAR(512) | Stores the S3 URL for the PAN card. || aadhaar_document_url | VARCHAR(512) | Stores the S3 URL for the Aadhaar card. || license_document_url | VARCHAR(512) | Stores the S3 URL for the license. || approval_status | ENUM('pending', 'approved', 'rejected') | DEFAULT 'pending' || rejection_reason | TEXT | Nullable. Filled by Admin if rejected. || profile_image_url | VARCHAR(512) | Nullable. For profile updates. || created_at | TIMESTAMP | Default: CURRENT_TIMESTAMP || updated_at | TIMESTAMP | Default: CURRENT_TIMESTAMP on update |

3. subscriptions

- Tracks the subscription status of each user. | Column Name | Data Type | Constraints & Notes | | :--- | :--- | | id | INT /

BIGINT | Primary Key, Auto Increment | | user_id | INT / BIGINT
| Foreign Key to users.id | | package_name | ENUM('silver',
'gold', 'platinum') | Not Null | | start_date | DATETIME | Not
Null | | end_date | DATETIME | Not Null | | status |
ENUM('active', 'expired')

4. payments

- Keeps a record of all transactions. | Column Name | Data Type | Constraints & Notes | | :--- | :--- | :--- |
- | id | INT / BIGINT | Primary Key, Auto Increment | | user_id | INT / BIGINT | Foreign Key to users.id | | subscription_id | INT / BIGINT | Foreign Key to subscriptions.id | | amount | DECIMAL(10, 2) | Not Null | | payment_gateway_txn_id | VARCHAR(255) | Transaction ID from the payment gateway. | | status | ENUM('success', 'failed', 'pending') | Not Null | | created_at | TIMESTAMP | Default: CURRENT_TIMESTAMP

Table Relationship:

- A user can have many subscriptions (one-to-many), though typically one active at a time.
- A subscription is linked to one payment.

3. API Endpoints & I/O

For this module, we will need approximately 5-6 core APIs.

1. Send OTP

- Endpoint: POST /api/auth/send-otp
- Input (Request Body):

JSON

{

```
"mobile_number": "9876543210"
```

```
}
```

- **Output (Success Response):**

JSON

```
{
```

```
"status": 200,
```

```
"message": "OTP sent successfully."
```

```
}
```

2. Verify OTP & Check User Status

- **Endpoint:** POST /api/auth/verify-otp
- **Input (Request Body):**

JSON

```
{
```

```
"mobile_number": "9876543210",
```

```
"otp": "123456"
```

```
}
```

- **Output (Case 1: New User):**

JSON

```
{
```

```
"status": 200,
```

```
"message": "OTP Verified. Proceed with registration.",
```

```
"is_new_user": true,
```

```
"registration_token": "temp_jwt_token_for_registration_steps"
```

```
}
```

- **Output (Case 2: Approved User - Login Success):**

JSON

```
{  
  "status": 200,  
  "message": "Login successful.",  
  "is_new_user": false,  
  "access_token": "full_jwt_auth_token",  
  "user_data": {  
    "id": 1,  
    "name": "John Doe",  
    "role": "retailer",  
    "approval_status": "approved"  
  }  
}
```

3. Register User (Submit Details & Documents)

- **Endpoint:** POST /api/auth/register
- **Note:** This will be a multipart/form-data request to handle file uploads.
- **Input (Form Data):**
 - name: "John Doe"
 - role: "retailer"
 - address: "123 Pharmacy Lane..."
 - pan_number: "ABCDE1234F"
 - aadhaar_number: "123456789012"

- **pharmacy_license_number:** "TS/123/XYZ"
- **pan_document:** (file)
- **aadhaar_document:** (file)
- **license_document:** (file)
- **package_name:** "gold"
- **payment_transaction_id:** "gateway_txn_id_from_frontend"
- **Output (Success Response):**

JSON

```
{  
  "status": 201,  
  "message": "Registration complete. Your application is under review.",  
  "user_id": 15,  
  "approval_status": "pending"  
}
```

4. Get Subscription Packages

- **Endpoint:** GET /api/packages
- **Input:** None
- **Output (Success Response):**

JSON

```
{  
  "status": 200,  
  "packages": [  
    { "id": 1, "name": "silver", "price": 1999, "features": [...] },  
    { "id": 2, "name": "gold", "price": 4999, "features": [...] },  
  ]  
}
```

```
{ "id": 3, "name": "platinum", "price": 9999, "features": [...] }  
]  
}
```

5. Get User Profile

- Endpoint: GET /api/user/me (Authenticated Route)
- Input: Auth Token in Header
- Output (Success Response):

JSON

```
{  
  "status": 200,  
  "user": {  
    "id": 15,  
    "name": "John Doe",  
    "mobile_number": "9876543210",  
    "role": "retailer",  
    // ... all other user fields  
  }  
}
```

6. Update User Profile

- Endpoint: PUT /api/user/me (Authenticated Route)
- Note: multipart/form-data if updating the profile image.
- Input (Request Body):

JSON

```
{
```

```
"name": "Johnathan Doe",  
"profile_image": "(file data if changed)"  
}
```

- **Output (Success Response):**

JSON

```
{  
  "status": 200,  
  "message": "Profile updated successfully.",  
  "user": { /* ... updated user object ... */ }  
}
```

Module 2: Admin Panel & Verification Module



Objective: To provide a secure and centralized interface for the platform administrator to manage users, verify registrations, and oversee platform content like advertisements and videos.

1. Detailed Workflow & Procedure

The admin panel is a separate, secure application (or section of the application) accessible only to authorized personnel.

Step 1: Admin Login

- An admin navigates to a dedicated login page (e.g., admin.medistocks.com).

- They log in using a secure email and password combination, not the OTP system used by regular users. This ensures a higher level of security for platform management.
- Upon successful login, they are granted an admin-specific access token and are redirected to the Admin Dashboard.

Step 2: Admin Dashboard Overview

- The dashboard is the landing page and provides a high-level "at-a-glance" view of the platform's status.
- It displays key metrics through widgets or cards:
 - New Pending Verifications: A count of users awaiting approval.
 - Total Users: A breakdown of registered Retailers vs. Distributors.
 - Pending Ad Requests: A count of advertisements awaiting approval.
 - Active Subscriptions: Total number of active paying users.
 - Platform Revenue: Basic revenue metrics (optional but valuable).

Step 3: User Verification Process

- The admin navigates to the "User Management" or "Pending Verifications" section from the dashboard or a sidebar menu.
- They are presented with a table or list of all users whose `approval_status` is pending.
- The admin clicks on a specific user to view their complete profile. This "detail view" shows:
 - All the information submitted during registration (Name, Role, PAN No, etc.).

- Clickable links to the documents stored on AWS S3 (PAN Card, Aadhaar, License). Clicking a link opens the document in a new tab for review.
- After reviewing the details and documents, the admin has two choices:
 - Approve: The admin clicks an "Approve" button. The system updates the user's approval_status from 'pending' to 'approved'. A confirmation notification (e.g., SMS, Push Notification, or Email) is automatically sent to the user informing them that their account is now active.
 - Reject: The admin clicks a "Reject" button. A modal window appears, requiring the admin to enter a reason for rejection (e.g., "PAN card image is not clear," "License number provided does not match the uploaded document"). The system updates the user's approval_status to 'rejected' and saves the reason in the rejection_reason field. A notification is sent to the user with the specific reason for rejection.

Step 4: Content Management (Videos)

- The admin can access a "Content" or "Videos" section.
- Here, they can upload new videos, add a title and description, and publish them to the platform.
- These published videos will then become visible to all approved Retailers and Distributors in their respective apps (as detailed in Module 7).

2. Database Schema

For this module, we will primarily interact with the users table from Module 1 and introduce two new tables: admins and videos.

1. users (Interaction)

- The admin panel's main function is to update the following fields in this existing table:
 - approval_status: The admin changes this from 'pending' to 'approved' or 'rejected'.
 - rejection_reason: The admin fills this field when a user is rejected.

2. admins (New Table)

- This table securely stores credentials for the admin users, keeping them separate from the main user base. | Column Name | Data Type | Constraints & Notes | | :--- | :--- | :--- | | id | INT / BIGINT | Primary Key, Auto Increment | | name | VARCHAR(255) | Not Null | | email | VARCHAR(255) | UNIQUE, Not Null. Used for login. | | password_hash | VARCHAR(255) | Not Null. Never store plain text passwords. | | role | ENUM('super_admin', 'verifier') | For role-based access control within the admin panel. | | created_at | TIMESTAMP | Default: CURRENT_TIMESTAMP |

3. videos (New Table)

- This table will store information about the videos uploaded by the admin. | Column Name | Data Type | Constraints & Notes | | :--- | :--- | :--- | | id | INT / BIGINT | Primary Key, Auto Increment | | title | VARCHAR(255) | Not Null | | description | TEXT | Nullable | | video_url | VARCHAR(512) | Not Null. The URL of the video file on S3. | | thumbnail_url | VARCHAR(512) | Nullable. URL for a video thumbnail image. | | posted_by_admin_id | INT / BIGINT | Foreign Key to admins.id | | created_at | TIMESTAMP | Default: CURRENT_TIMESTAMP |
-

3. API Endpoints & I/O

These APIs are for the admin panel frontend and must be protected by an admin-only authentication middleware.

1. Admin Login

- Endpoint: POST /api/admin/login
- Input (Request Body):

JSON

```
{  
  "email": "admin@medistocks.com",  
  "password": "secure_password"  
}
```

- Output (Success Response):

JSON

```
{  
  "status": 200,  
  "message": "Admin login successful.",  
  "token": "admin_jwt_auth_token",  
  "admin": { "id": 1, "name": "Super Admin", "email":  
    "admin@medistocks.com" }  
}
```

2. Get Dashboard Statistics

- Endpoint: GET /api/admin/dashboard/stats
- Input: Admin Auth Token in Header
- Output (Success Response):

JSON

```
{  
  "status": 200,  
  "stats": {  
    "pending_verifications": 15,  
    "total_retailers": 150,  
    "total_distributors": 35,  
    "pending_ad_requests": 5  
  }  
}
```

3. Get List of Users for Verification

- Endpoint: GET /api/admin/users?status=pending
- Input: Admin Auth Token in Header. Can use query parameters for filtering (e.g., ?status=pending, ?status=approved) and pagination (?page=1&limit=10).
- Output (Success Response):

JSON

```
{  
  "status": 200,  
  "users": [  
    { "id": 15, "name": "John Doe", "role": "retailer", "created_at": "..."},  
    { "id": 16, "name": "Jane Smith", "role": "distributor", "created_at":  
      "..."}  
  ],  
  "pagination": { "total_pages": 5, "current_page": 1 }  
}
```

4. Get Full User Details for Review

- Endpoint: GET /api/admin/users/{userId}
- Input: Admin Auth Token in Header, userId from URL.
- Output (Success Response):

JSON

```
{  
  "status": 200,  
  "user": {  
    "id": 15,  
    "name": "John Doe",  
    // ... all fields from the users table, including document URLs  
    "pan_document_url": "https://s3...",  
    "aadhaar_document_url": "https://s3...",  
    "license_document_url": "https://s3..."  
  }  
}
```

5. Update User Verification Status

- Endpoint: PUT /api/admin/users/{userId}/status
- Input (Request Body for Approval):

JSON

```
{  
  "status": "approved"  
}
```

- Input (Request Body for Rejection):

JSON

```
{  
  "status": "rejected",  
  "reason": "The PAN card number did not match the uploaded  
document."  
}
```

- **Output (Success Response):**

JSON

```
{  
  "status": 200,  
  "message": "User status has been updated successfully."  
}
```

6. Post a New Video

- **Endpoint:** POST /api/admin/videos
- **Input:** multipart/form-data containing title, description, and the video file.
- **Output (Success Response):**

JSON

```
{  
  "status": 201,  
  "message": "Video uploaded successfully.",  
  "video": { /* ... new video object ... */ }  
}
```

Module 2: Admin Panel & Verification Module (Enhanced)

Objective: To provide a secure and centralized interface for the platform administrator to manage users, verify registrations, monitor subscriptions and revenue, and oversee platform content.

1. Detailed Workflow & Procedure

Step 1: Admin Login

- An admin logs in via a secure email and password page.
- Upon success, they are redirected to the Admin Dashboard.

Step 2: Admin Dashboard Overview

- The dashboard provides a comprehensive at-a-glance view of the platform's key performance indicators (KPIs):
 - **User Metrics:**
 - New Pending Verifications: A direct link to the verification queue.
 - Total Users: A count broken down by Retailers vs. Distributors.
 - **Subscription Metrics:**
 - Active Subscriptions: Total number of users with a current, active plan.
 - Subscriptions Nearing Expiry: A count of subscriptions expiring in the next 7 days.
 - **Financial Metrics:**
 - Total Revenue (All Time): A gross revenue figure.
 - Revenue (Last 30 Days): A more immediate performance indicator.
 - **Content Metrics:**

- **Pending Ad Requests:** A count of advertisements awaiting approval.

Step 3: User Verification Process

- (This workflow remains the same as previously described: Admin reviews pending users, views their documents, and either Approves or Rejects with a reason.)

Step 4: Comprehensive User & Subscription Management

- The admin navigates to a "Users" section, which displays a table of all registered users (not just pending ones).
- **Features of this section:**
 - **Search:** Find users by name, mobile number, or license number.
 - **Filtering:** Filter the list by Role (Retailer/Distributor), Approval Status (Pending/Approved/Rejected), and Subscription Plan (Silver/Gold/Platinum).
 - **Viewing Details:** Clicking any user reveals their full profile, including their subscription status (Active, Expired), plan name, and expiry date.
 - **Actions:** For each approved user, an admin can perform actions such as:
 - **Deactivate Account:** Temporarily suspend a user's access to the platform.
 - **View Payment History:** See a list of all successful payments made by the user.
 - **Manually Extend Subscription:** In special cases (e.g., as a goodwill gesture), an admin can add a few grace days to a user's active subscription.

Step 5: Revenue & Analytics

- A dedicated "Revenue" or "Analytics" section provides insights into the platform's financial health.
- **Workflow:**
 - The admin accesses this section to view financial data, which is aggregated from successful transactions in the payments table.
 - **Visualizations:** The page will feature interactive charts:
 - A line chart displaying revenue trends over a selectable period (e.g., last 30 days, last 6 months, year-to-date).
 - A pie chart showing the revenue breakdown by subscription package (e.g., 50% from Gold, 30% from Platinum, 20% from Silver).
 - The admin can export this data (e.g., as a CSV file) for offline analysis.

Step 6: Content Management (Videos)

- (This workflow remains the same: Admin can upload, manage, and publish videos to the platform.)

2. Database Schema

We will use the previously defined tables (admins, users, subscriptions, payments, videos). I am adding one field to the users table to support the new deactivation feature.

1. users (Enhancement)

- To manage account deactivation, we will add a new boolean field.
- | Column Name | Data Type | Constraints & Notes |
|-------------|-----------|--|
| is_active | BOOLEAN | DEFAULT TRUE. Admin can set this to FALSE to suspend an account. |

2. Schema Usage for New Features:

- **Total User Management:** The users table is the primary source. The is_active flag controls access. The subscriptions table is joined to show plan details.
 - **Revenue Analytics:** All calculations are performed by aggregating data from the payments table where status = 'success'. The query will SUM(amount) and group results by date or subscription type.
-

3. API Endpoints & I/O (Expanded)

Here are the additional endpoints required for the enhanced features, all protected by admin authentication.

1. Get All Users (with Filtering)

- **Endpoint:** GET /api/admin/users
- **Purpose:** Powers the main user management table.
- **Query Parameters for Filtering:**
 - ?status=approved
 - ?role=retailer
 - ?package=gold
 - ?search=JohnDoe
 - ?page=1&limit=20
- **Output (Success Response):** A paginated list of user objects with relevant subscription data joined.

2. Deactivate/Reactivate a User Account

- **Endpoint:** PUT /api/admin/users/{userId}/activation
- **Input (Request Body):**

JSON

```
{  
  "is_active": false  
}
```

- **Output (Success Response):**

JSON

```
{  
  "status": 200,  
  "message": "User account has been deactivated."  
}
```

3. Manually Extend a User's Subscription

- **Endpoint:** POST /api/admin/users/{userId}/subscription/extend
- **Input (Request Body):**

JSON

```
{  
  "days": 7,  
  "reason": "Customer support goodwill gesture."  
}
```

- **Output (Success Response):**

JSON

```
{  
  "status": 200,  
  "message": "Subscription extended by 7 days.",  
  "new_expiry_date": "2025-10-25T12:00:00Z"  
}
```

4. Get Revenue Summary (for Dashboard Widgets)

- Endpoint: GET /api/admin/revenue/summary
- Input: None
- Output (Success Response):

JSON

```
{  
  "status": 200,  
  "summary": {  
    "total_revenue": 750000.00,  
    "revenue_last_30_days": 85000.00,  
    "active_subscriptions": 185,  
    "subscriptions_nearing_expiry": 12  
  }  
}
```

5. Get Revenue Time-Series Data (for Line Chart)

- Endpoint: GET /api/admin/revenue/timeseries?range=30d
- Query Parameters: range can be 30d, 6m, 1y.
- Output (Success Response):

JSON

```
{  
  "status": 200,  
  "data": [  
    { "date": "2025-08-28", "revenue": 2000.00 },  
    { "date": "2025-08-29", "revenue": 3500.00 },  
  ]  
}
```

```
// ... more data points
```

```
]
```

```
}
```

6. Get Revenue Breakdown by Package (for Pie Chart)

- Endpoint: GET /api/admin/revenue/breakdown
- Input: None
- Output (Success Response):

JSON

```
{
```

```
  "status": 200,
```

```
  "breakdown": [
```

```
    { "package": "silver", "revenue": 150000.00 },
```

```
    { "package": "gold", "revenue": 400000.00 },
```

```
    { "package": "platinum", "revenue": 200000.00 }
```

```
  ]
```

```
}
```

Module 3: Product & Inventory Management

Module

Objective: To enable Distributors and Retailers to list near-expiry products for sale on a public marketplace, and to provide a private tool for them to manage their own personal stock.

This module is broken into two key features: the Public Marketplace and the Private "My Inventory".

1. Detailed Workflow & Procedure

A. The Public Marketplace

1. Posting a Product for Sale

- A logged-in and approved Distributor or Retailer accesses a "Sell Product" section in the app.
- They fill out a detailed product form. To ensure quality and trust, we will require comprehensive information:
 - **Product Name:** E.g., "Crocin Advance 500mg"
 - **Manufacturer:** E.g., "GSK"
 - **Product Image:** A clear image of the product packaging.
 - **Product Type:** A dropdown selection (e.g., Tablet, Capsule, Syrup, Injection, Ointment).
 - **Batch Number:** Crucial for traceability.
 - **Expiry Date:** A date picker is used to select the date. The system can validate that this date is in the near future.
 - **Quantity:** The number of units for sale (e.g., 100 strips, 50 bottles).
 - **MRP (Maximum Retail Price):** The original price per unit.
 - **Discounted Selling Price:** The price per unit offered on Medistocks. The UI will automatically calculate and display the discount percentage to attract buyers.
 - **Description:** Any additional notes or details.
- Upon submission, the product is listed and becomes visible to other users (primarily Retailers) on the marketplace. The product's location is automatically tied to the seller's registered address.

2. Viewing and Filtering Products (Retailer Experience)

- A Retailer navigates to the "Marketplace" or "Browse Products" screen.
- They see a feed of available products, which can be sorted (e.g., by Newest, Expiring Soonest, Highest Discount).
- Crucial Feature - Filtering: The retailer can use powerful filters to find exactly what they need:
 - Location-Based Filter: A primary filter to find "Nearby Inventory." The user can select a radius (e.g., Within 10km, 25km, 50km, 100km) to find sellers in their vicinity.
 - Category Filter: Filter by Product Type to find specific kinds of medicine.
 - Search Bar: Search directly by Product Name or Manufacturer.
- Clicking on a product opens a detailed view with all the information, images, and details about the seller (e.g., their business name and rating).

B. Private "My Inventory"

- This feature acts as a personal stock management tool for both Retailers and Distributors. It is private and separate from the public marketplace.
- Adding Stock: The user selects "My Inventory" and clicks "Add New Stock." They fill a simpler form:
 - Product Name, Manufacturer, Batch No., Expiry Date, Quantity.
 - Value-Add Field: Purchase Source (to track where they got the stock from) and Purchase Date.
- Inventory Dashboard: The user can view their entire stock list, sorted by expiry date.

- Value-Add Feature - Expiry Alerts: The system can automatically send notifications to the user when a product in their private inventory is about to expire (e.g., 60 days left), suggesting they "List it on the Marketplace" with a single click, which would pre-fill the selling form for them.

2. Database Schema

For this module, we need two new tables and must enhance the existing users table.

1. users (Enhancement)

- To enable the location-based search, we will geocode the user's address upon registration and store their coordinates. | Column Name | Data Type | Constraints & Notes | | :--- | :--- | :--- | | business_name | VARCHAR(255) | Recommended. More professional than a personal name. | | latitude | DECIMAL(10, 8) | Nullable. Stores the geographic latitude. | | longitude | DECIMAL(11, 8) | Nullable. Stores the geographic longitude. |

2. products (New Table for Marketplace)

- This table stores all items listed for public sale. | Column Name | Data Type | Constraints & Notes | | :--- | :--- | :--- | | id | INT / BIGINT | Primary Key, Auto Increment | | seller_id | INT / BIGINT | Foreign Key to users.id | | product_name | VARCHAR(255) | Not Null | | manufacturer | VARCHAR(255) | Nullable | | image_url | VARCHAR(512) | Not Null. S3 URL of the product image. | | product_type | ENUM(...) | ('tablet', 'syrup', 'injection', 'ointment', 'capsule', 'other') | | batch_number | VARCHAR(50) | Not Null | | expiry_date | DATE | Not Null | | quantity | INT | Not Null | | mrp | DECIMAL(10, 2) | Not Null. Maximum Retail Price. | | discount_price | DECIMAL(10, 2) | Not Null. The actual selling

price. | | description | TEXT | Nullable | | status | ENUM(...) |
DEFAULT 'available'. ('available', 'sold', 'expired', 'delisted') | |
created_at | TIMESTAMP | Default: CURRENT_TIMESTAMP | |
updated_at | TIMESTAMP | Default: CURRENT_TIMESTAMP on
update |

3. user_inventory (New Table for Private Stock)

- Stores the private stock records for each user. | Column Name |
Data Type | Constraints & Notes | | :--- | :--- | :--- | | id | INT /
BIGINT | Primary Key, Auto Increment | | user_id | INT / BIGINT
| Foreign Key to users.id | | product_name | VARCHAR(255) |
Not Null | | manufacturer | VARCHAR(255) | Nullable | |
batch_number | VARCHAR(50) | Not Null | | expiry_date |
DATE | Not Null | | quantity | INT | Not Null | | purchase_date
| DATE | Nullable | | purchase_source | VARCHAR(255) |
Nullable | | created_at | TIMESTAMP | Default:
CURRENT_TIMESTAMP |

3. API Endpoints & I/O

All these routes are authenticated and accessible only to approved users.

Marketplace APIs

1. Create Product Listing

- Endpoint: POST /api/products
- Input: multipart/form-data containing all fields from the products table and the image file.
- Output (Success): The newly created product JSON object.

2. Get All Product Listings

- Endpoint: GET /api/products
- Query Parameters:

- ?distance_km=25 (Backend uses user's stored lat/long to perform a geospatial query)
- ?type=syrup
- ?search=Calpol
- ?sort_by=expiry_asc
- ?page=1&limit=15
- **Output (Success):** A paginated JSON array of product listings.

3. Get Single Product Details

- **Endpoint:** GET /api/products/{productId}
- **Input:** productId from URL.
- **Output (Success):** A single detailed JSON object for the product, including seller info (business_name, seller_id).

Private Inventory APIs

4. Add Item to My Inventory

- **Endpoint:** POST /api/inventory
- **Input (Request Body):** JSON with fields from the user_inventory table.
- **Output (Success):** The new inventory item JSON object.

5. Get My Inventory Items

- **Endpoint:** GET /api/inventory
- **Query Parameters:** ?sort_by=expiry_asc
- **Output (Success):** A JSON array of the user's private inventory items.

6. Update/Delete My Inventory Item

- **Endpoint:** PUT /api/inventory/{itemId} or DELETE /api/inventory/{itemId}
- **Input:** itemId from URL. PUT requires a request body with fields to update.
- **Output (Success):** An updated item object or a success message.

Module 4: Order & Transaction Management

Module

Objective: To provide a structured and secure workflow for Retailers to request products, for Distributors to manage these requests, and for both parties to communicate and finalize the transaction through an in-app system.

1. Detailed Workflow & Procedure

The entire process is managed by changing the status of an order, creating a clear and trackable lifecycle for each transaction.

Step 1: Retailer Sends a Purchase Request

- A logged-in Retailer browses the marketplace and finds a product they want.
- On the product details page, they click the "Send Purchase Request" button.
- A prompt appears asking for the desired quantity. The system validates that the requested quantity is not more than the available stock.
- The Retailer confirms the request.

- **System Action:** A new entry is created in the orders table with a status of pending. The requested quantity of the product is temporarily reserved. The Distributor (seller) receives an in-app notification about the new request.

Step 2: Distributor Responds to the Request

- The Distributor sees the new request in their "Incoming Orders" dashboard.
- They can view the order details, including the Retailer's business name and the requested product/quantity.
- **The Distributor has two options:**
 - **Reject:** They can reject the request. The order status changes to rejected. The reserved stock is released. The Retailer is notified that their request was not accepted.
 - **Accept:** They accept the request. The order status changes to accepted.
- **System Action:** Upon acceptance, two things happen immediately:
 1. The Retailer is notified that their request has been accepted.
 2. An in-app chat is now enabled *exclusively* between this Retailer and Distributor, linked to this specific order.

Step 3: Negotiation via In-App Chat

- Both parties can now communicate directly within the order's chat screen.
- As per the business logic, they can negotiate final prices, discuss logistics (pickup/delivery times), and agree on payment terms (e.g., Cash on Delivery). The platform's role here is only to facilitate this communication. All chat history is saved.

Step 4: Invoice Generation and Verification Code

- After negotiations are complete, the Distributor clicks a "Generate Invoice" button within the order details.
- **System Action:**
 1. A basic invoice is generated.
 2. A unique 6-digit Verification Code is created and securely associated with the order.
 3. The order status is updated to awaiting_confirmation.
 4. The Verification Code is sent only to the Retailer via SMS and an in-app notification. The Distributor cannot see this code. This is a crucial security step.

Step 5: Transaction Completion

- This step typically happens in person during the exchange of goods.
- The Retailer provides the 6-digit Verification Code to the Distributor.
- The Distributor enters this code into the order screen in their app and submits it.
- System Action: The system validates the code.
 - **If Correct:** The order status is updated to completed. The transaction is officially finished.
 - **If Incorrect:** An error message is displayed.
- Value-Add Feature - Ratings & Reviews: After an order is completed, both the Retailer and Distributor are prompted to leave a rating (1-5 stars) and a review for each other. This builds trust within the platform.

2. Database Schema

This workflow requires several new tables to manage orders, communication, and trust.

1. orders (New Table)

- The central table tracking every transaction. | Column Name | Data Type | Constraints & Notes | | :--- | :--- | :--- | | id | INT / BIGINT | Primary Key, Auto Increment | | product_id | INT / BIGINT | Foreign Key to products.id | | buyer_id | INT / BIGINT | Foreign Key to users.id (The Retailer) | | seller_id | INT / BIGINT | Foreign Key to users.id (The Distributor) | | requested_quantity | INT | Not Null | | status | ENUM(...) | ('pending', 'accepted', 'rejected', 'awaiting_confirmation', 'completed', 'cancelled') | | verification_code_hash | VARCHAR(255) | Stores a hashed version of the 6-digit code for secure verification. | | created_at | TIMESTAMP | Default: CURRENT_TIMESTAMP | | updated_at | TIMESTAMP | Default: CURRENT_TIMESTAMP on update |

2. chat_messages (New Table)

- Stores the conversation history for each order. | Column Name | Data Type | Constraints & Notes | | :--- | :--- | :--- | | id | INT / BIGINT | Primary Key, Auto Increment | | order_id | INT / BIGINT | Foreign Key to orders.id. Scopes the chat to one transaction. | | sender_id | INT / BIGINT | Foreign Key to users.id | | message_body | TEXT | Not Null | | is_read | BOOLEAN | DEFAULT FALSE | | sent_at | TIMESTAMP | Default: CURRENT_TIMESTAMP |

3. ratings (New Table - Value Add)

- To build a reputation system. | Column Name | Data Type | Constraints & Notes | | :--- | :--- | :--- | | id | INT / BIGINT | Primary Key, Auto Increment | | order_id | INT / BIGINT | Foreign Key to orders.id | | rated_by_user_id | INT / BIGINT | FK to users.id (The user giving the rating) | | rated_user_id | INT / BIGINT | FK to users.id (The user receiving the rating) |

rating_value | TINYINT | 1 to 5 | | comment | TEXT | Nullable |
| created_at | TIMESTAMP | Default: CURRENT_TIMESTAMP |

3. API Endpoints & I/O

1. Create a Purchase Request

- Endpoint: POST /api/orders
- Role: Retailer
- Input (Body):

JSON

```
{ "product_id": 123, "quantity": 50 }
```

- Output (Success): The new order object with status: "pending".

2. Get My Orders

- Endpoint: GET /api/orders?as=buyer or GET /api/orders?as=seller
- Role: Both
- Output (Success): A paginated list of the user's orders.

3. Respond to a Purchase Request

- Endpoint: PUT /api/orders/{orderId}/status
- Role: Distributor
- Input (Body):

JSON

```
{ "status": "accepted" } // or { "status": "rejected" }
```

- Output (Success): A message confirming the update.

4. Chat APIs (Real-time Communication)

- Note: While REST can be used, this is a prime use case for WebSockets for a seamless, real-time chat experience.

- **Get Chat History:** GET `/api/orders/{orderId}/chat`
- **Send Message:** POST `/api/orders/{orderId}/chat` (or via WebSocket event)
- **Role:** Both (only if part of the order)

5. Generate Invoice / Get Verification Code

- **Endpoint:** POST `/api/orders/{orderId}/initiate-completion`
- **Role:** Distributor
- **Output (Success):** A message confirming the invoice is generated and the code has been sent to the buyer.

6. Verify & Complete Transaction

- **Endpoint:** POST `/api/orders/{orderId}/complete`
- **Role:** Distributor
- **Input (Body):**

JSON

```
{ "verification_code": "123456" }
```

- **Output (Success):**

JSON

```
{ "status": 200, "message": "Transaction completed successfully!" }
```

7. Submit a Rating

- **Endpoint:** POST `/api/orders/{orderId}/rate`
- **Role:** Both (after completion)
- **Input (Body):**

JSON

```
{ "rating": 5, "comment": "Great seller, fast delivery." }
```

- **Output (Success):** A confirmation message.

Module 5: AI-Powered Demand Forecasting

Module

Objective: To provide actionable, data-driven insights to both Retailers and Distributors, suggesting which products are likely to see a surge in demand based on weather, season, and location. This helps Retailers optimize stock and helps Distributors focus their sales efforts.

1. Detailed Workflow & Procedure

This module works differently from the others. It's a data-processing and suggestion engine that runs primarily in the background.

A. User Experience

- A logged-in Retailer or Distributor will see a special widget on their main Dashboard called "Demand Spotlight" or "Trending in Your Area".
- This widget will display concise, easy-to-understand insights. For example:

"Heatwave Alert for Hyderabad! Weather forecasts predict rising temperatures. Expect a ~20% increase in demand for oral rehydration salts (ORS) and skin-care products this week. Stock Up Now."

- Each insight will be actionable. The "Stock Up Now" text would be a link that takes the user to the marketplace, pre-filtered for the suggested product categories.
- The insights are personalized based on the user's registered location. A user in Chennai might see different suggestions than a user in Delhi on the same day.

B. System's Backend Process

This is a recurring, automated process that does not require user interaction.

Step 1: Data Aggregation (Runs Automatically)

- A scheduled background job (e.g., running once every 24 hours) collects data from several sources:
 1. **Internal Sales History:** It analyzes the orders table to identify historical sales patterns. For example, it checks which products sold most in Hyderabad during September of last year.
 2. **External Weather API:** It connects to a third-party weather service (like OpenWeatherMap) to fetch current and future weather forecasts (e.g., temperature, humidity, precipitation) for key operational areas (like Hyderabad, Bengaluru, etc.).
 3. **Seasonal & Event Data:** The system is aware of the current date and can map it to seasons (e.g., Winter, Monsoon, Summer) or known local events (e.g., festival seasons).

Step 2: The "AI" Analysis Engine

- The collected data is fed into an analysis engine. This can be implemented in two phases:
 - **Phase 1 (Rule-Based System):** A simpler, effective starting point. The system uses a set of predefined rules. For example:
 - IF season is 'Monsoon' AND rainfall forecast > 10mm, THEN increase demand prediction for 'Cold & Flu' and 'Antifungal' categories.
 - IF temperature forecast > 40°C, THEN increase demand prediction for 'ORS' and 'Sunscreen' categories.
 - **Phase 2 (Machine Learning Model):** A more advanced implementation. A Machine Learning model is trained on

historical sales, weather, and seasonal data. This model can learn complex patterns and make more accurate predictions about the percentage increase in demand for specific product types.

Step 3: Storing the Insights

- The output of the analysis is a set of actionable insights. These insights are not generated in real-time when a user logs in.
- Instead, they are pre-calculated and stored in a new `demand_forecasts` table. This ensures the dashboard loads instantly for the user.

2. System Architecture & Database Schema

This module relies more on a data processing architecture than simple CRUD operations.

***Caption:** Data from the Medistocks DB and a Weather API is fed into a Processing Engine, which saves insights into a Forecasts table for fast API access.*

`demand_forecasts` (New Table)

This table stores the pre-computed results of the analysis engine.

Column Name	Data Type	Constraints & Notes
<code>id</code>	<code>INT / BIGINT</code>	Primary Key, Auto Increment
<code>location_city</code>	<code>VARCHAR(100)</code>	e.g., 'Hyderabad'. Used for targeting the insight.
<code>location_state</code>	<code>VARCHAR(100)</code>	e.g., 'Telangana'

Column Name	Data Type	Constraints & Notes
forecast_period_start	DATE	The date this forecast becomes valid.
forecast_period_end	DATE	The date this forecast expires.
triggering_factor	VARCHAR(255)	A short description of the cause, e.g., 'Monsoon Season', 'Heatwave Forecast'.
suggested_product_categories	JSON or TEXT	Stores an array of suggested categories, e.g., ["ORS", "Skin Care"].
demand_increase_prediction	INT	The predicted percentage increase, e.g., 20 for 20%.
insight_title	VARCHAR(255)	The headline for the widget, e.g., "Heatwave Alert for Hyderabad!".
insight_text	TEXT	The descriptive text shown to the user.
created_at	TIMESTAMP	Default: CURRENT_TIMESTAMP

Export to Sheets

3. API Endpoints & I/O

The user-facing side of this module is very simple, requiring only one API endpoint.

1. Get Demand Forecast for User's Dashboard

- **Endpoint:** GET /api/dashboard/forecasts
- **Role:** Retailer, Distributor
- **Backend Logic:**
 1. The API identifies the logged-in user and retrieves their registered city (e.g., 'Hyderabad') from their profile.
 2. It then queries the demand_forecasts table to find the most recent, active forecast for that city where the current date is between forecast_period_start and forecast_period_end.
- **Input:** None (User info is derived from the authentication token).
- **Output (Success Response):**

JSON

```
{
  "status": 200,
  "forecasts": [
    {
      "id": 1,
      "title": "Heatwave Alert for Hyderabad!",
      "text": "Weather forecasts predict rising temperatures. Expect a ~20% increase in demand for oral rehydration salts (ORS) and skin-care products this week.",
      "suggested_categories": ["ORS", "Skin Care"],
      "valid_until": "2025-10-04"
    }
  ]
}
```

- If no relevant forecast is found for the user's location, it returns an empty array.

Module 6: Advertisement Management

Module

Objective: To create a self-service advertising platform where Distributors and Retailers can purchase ad space to promote their business or products. This process will be overseen by an Admin to ensure ad quality and relevance.

1. Detailed Workflow & Procedure

The lifecycle of an ad involves three main stages: creation by the user, approval by the admin, and display by the system.

Stage 1: User Creates an Ad Campaign

- A logged-in user (Retailer or Distributor) navigates to the "Advertise" section of the app.
- **They initiate a new ad campaign by filling out a creation form:**
 - **Campaign Name:** A private name for their reference (e.g., "Monsoon Sale 2025").
 - **Ad Banner Image:** Upload a high-quality banner image.
 - **Headline:** A short, catchy title for the ad.
 - **Description:** A brief text detailing the offer.
 - **Value-Add Feature - Call-to-Action (CTA) Link:** The user provides a link for where the ad should redirect upon being clicked (e.g., their Medistocks profile page, or a specific product they have listed).

- **Placement & Duration:** The user selects where the ad will be shown (e.g., "Dashboard Banner") and for how long (e.g., 7 days, 15 days). The system displays the price, which is calculated based on these selections.
- **Payment:** The user is directed to the payment gateway to pay the calculated amount.
- **System Action:** Upon successful payment, a new record is created in the advertisements table with a status of pending_approval. The user receives a confirmation that their ad has been submitted for review.

Stage 2: Admin Reviews and Manages the Ad

- The Admin sees the new ad request in the "Ad Management" section of their admin panel (as designed in Module 2).
- The Admin reviews the ad content (image, text, and link) to ensure it meets the platform's community guidelines.
- **The Admin has two options:**
 - **Approve:** The ad status is updated to approved. The system sets the start_date to the current time and calculates the end_date based on the purchased duration. The ad is now live.
 - **Reject:** The ad status is updated to rejected. The Admin must provide a reason for the rejection (e.g., "Low-quality image," "Misleading claim"). The user is notified, and a refund process can be initiated.

Stage 3: System Displays the Live Ad

- The main application (e.g., the Retailer dashboard) has a designated ad slot.
- When a user loads a page with an ad slot, the system's frontend makes an API call to fetch a live ad.

- **Backend Logic:** The system queries the advertisements table for an entry where:
 - status is approved.
 - The current date/time is between the start_date and end_date.
- If multiple ads are active, the system can rotate them randomly to ensure fair exposure.
- Value-Add Feature - Ad Analytics: Each time an ad is displayed (an impression) or clicked, an event is logged. This allows the advertiser to see basic performance metrics for their campaign.

2. Database Schema

We will need two new tables to manage the ads and their performance.

1. advertisements (New Table)

- This table stores all information related to an ad campaign. |
 Column Name | Data Type | Constraints & Notes | | :--- | :--- | :--- |
 | id | INT / BIGINT | Primary Key, Auto Increment | |
 user_id | INT / BIGINT | Foreign Key to users.id (The advertiser) | |
 | campaign_name | VARCHAR(255) | Not Null | |
 banner_image_url | VARCHAR(512) | Not Null, S3 URL | |
 headline | VARCHAR(255) | Not Null | | description | TEXT |
 Nullable | | cta_link | VARCHAR(512) | The URL the ad points to. | | status | ENUM(...) | DEFAULT 'pending_approval'. ('pending_approval', 'approved', 'rejected', 'expired') | |
 placement | ENUM(...) | DEFAULT 'dashboard_banner'. ('dashboard_banner', 'search_banner') | |
 requested_duration_days | INT | Not Null | | start_date | DATETIME | Nullable. Set on approval. | | end_date | DATETIME | Nullable. Set on approval. | | amount_paid | DECIMAL(10, 2) | Not Null | | payment_transaction_id |

VARCHAR(255) | Not Null | | rejection_reason | TEXT | Nullable.
Filled by Admin on rejection. | | created_at | TIMESTAMP |
Default: CURRENT_TIMESTAMP |

2. ad_analytics (New Table - Value Add)

- Logs impression and click events for basic performance tracking.
| Column Name | Data Type | Constraints & Notes | | :--- | :--- |
:--- | | id | INT / BIGINT | Primary Key, Auto Increment | | ad_id
| INT / BIGINT | Foreign Key to advertisements.id | |
event_type | ENUM('impression', 'click') | Not Null | |
viewer_user_id | INT / BIGINT | Foreign Key to users.id (Who
saw/clicked the ad) | | timestamp | TIMESTAMP | Default:
CURRENT_TIMESTAMP |

3. API Endpoints & I/O

The APIs are split between user-facing, system-facing, and admin-facing roles.

User-Facing APIs

1. Create New Ad Request

- Endpoint: POST /api/advertisements
- Role: Retailer, Distributor
- Input: multipart/form-data with campaign details, placement, duration, and the banner image file.
- Output (Success): The new ad object with status: "pending_approval".

2. View My Ad Campaigns

- Endpoint: GET /api/advertisements/my-ads
- Role: Retailer, Distributor

- **Output (Success):** A list of the user's ad campaigns, their statuses, and basic analytics (e.g., total impressions and clicks, aggregated from the `ad_analytics` table).

System-Facing API

3. Fetch an Active Ad for Display

- **Endpoint:** `GET /api/advertisements/active`
- **Role:** System (called by the frontend)
- **Output (Success):** A single, randomly selected active ad object to be displayed in the UI.

4. Track an Ad Event

- **Endpoint:** `POST /api/advertisements/{adId}/track`
- **Role:** System (called by the frontend)
- **Input (Body):**

JSON

`{ "event": "impression" } // or { "event": "click" }`

- **Output (Success):** 200 OK status.

Admin-Facing APIs

5. Update Ad Status (Approve/Reject)

- **Endpoint:** `PUT /api/admin/ads/{adId}/status`
- **Role:** Admin
- **Input (Body):**

JSON

`{ "status": "approved" } // or { "status": "rejected", "reason": "Ad image is blurry." }`

- **Output (Success):** A confirmation message.

Module 7: Content & Notifications Module

Objective: To provide a channel for admins to share video content with users, and to implement a robust, platform-wide notification system that keeps users informed about important events and actions.

This module is divided into two parts: the Video Content Platform and the Centralized Notification System.

1. Detailed Workflow & Procedure

A. Video Content Platform

1. Admin Workflow (Content Creation)

- As defined in Module 2, an Admin uses their control panel to manage video content.
- They can upload a video file (which is stored on AWS S3), add a title, a description, and a thumbnail image, and then publish it.
- Published videos are immediately available to all approved users on the platform.

2. User Workflow (Content Consumption)

- A logged-in Retailer or Distributor can access a "Videos" or "Learning Center" section within the app.
- They are presented with a feed of all published videos, typically sorted with the newest ones first.
- The user can tap on any video to watch it in a native video player.
- Value-Add Feature: The video feed can be enhanced with categories (e.g., "Platform News," "Industry Updates," "Sales Tutorials") and a search bar to help users find relevant content easily.

B. Centralized Notification System

This is a background system responsible for generating and delivering alerts to users based on specific events.

1. Notification Triggers

- Notifications are automatically generated by the system when key events occur across different modules:
 - **User Onboarding:** Account is approved or rejected.
 - **Transactions:** A new purchase request is received; a request is accepted or rejected; an invoice is generated and a verification code is ready.
 - **Communication:** A new chat message is received.
 - **Inventory:** A product in a user's private "My Inventory" is nearing its expiry date.
 - **Content:** A new video is published by the admin.
 - **Advertisements:** An ad campaign is approved or rejected.

2. Delivery Channels

- The system uses multiple channels to ensure users receive timely information:
 1. **In-App Notification Center:** A bell icon  in the app's main interface. It will show a badge with the count of unread notifications. Users can tap it to see a history of all their notifications.
 2. **Push Notifications:** Instant alerts sent directly to the user's mobile device for high-priority events like new purchase requests and chat messages. This requires user permission and integration with services like Firebase Cloud Messaging (FCM).
 3. **SMS:** Reserved for the most critical alerts, such as the transaction verification code.

3. User Workflow

- The user receives a push notification on their phone for an important event.
- Inside the app, they can access the Notification Center to review all past alerts.
- Tapping on a notification will deep-link them to the relevant screen.
For example:
 - Tapping a "New Chat Message" alert opens the specific chat conversation for that order.
 - Tapping an "Account Approved" alert takes them to their dashboard.

2. Database Schema

This module requires a table for notifications and a small addition to the users table. The videos table was defined in Module 2.

1. users (Enhancement)

- To send push notifications, we must store a unique device token for each user session. | Column Name | Data Type | Constraints & Notes | | :--- | :--- | :--- | | device_token | VARCHAR(255) | Nullable. Stores the FCM/APNs token from the user's mobile device. |

2. notifications (New Table)

- This table acts as a persistent log of all notifications sent to users. | Column Name | Data Type | Constraints & Notes | | :--- | :--- | :--- | | id | INT / BIGINT | Primary Key, Auto Increment | | user_id | INT / BIGINT | Foreign Key to users.id (The recipient) | | notification_type | ENUM(...) | ('account_status', 'new_order', 'new_message', 'order_update', 'inventory_alert', 'new_video', 'ad_status') | |

title | VARCHAR(255) | Not Null (e.g., "New Purchase Request") | | body | TEXT | The main content of the notification. | | related_entity_id | INT | Nullable. Used for deep-linking. E.g., the order_id or video_id. | | is_read | BOOLEAN | DEFAULT FALSE | | created_at | TIMESTAMP | Default: CURRENT_TIMESTAMP |

3. API Endpoints & I/O

The creation of notifications is a backend process, not a direct API call. The APIs for this module are primarily for users to retrieve content and manage their notifications.

Video APIs

1. Get All Videos

- **Endpoint: GET /api/videos**
- **Role: Retailer, Distributor**
- **Output (Success): A paginated list of video objects (id, title, thumbnail_url, description, created_at).**

Notification APIs

2. Get My Notifications

- **Endpoint: GET /api/notifications**
- **Role: Retailer, Distributor**
- **Output (Success): A list of the user's notifications, ordered from newest to oldest.**

3. Mark Notifications as Read

- **Endpoint: POST /api/notifications/mark-read**
- **Role: Retailer, Distributor**

- **Input (Body):** Can be empty to mark all as read, or specific.

JSON

```
{ "notification_ids": [101, 102] }
```

- **Output (Success):** A confirmation message.

4. Register Device for Push Notifications

- **Endpoint:** POST /api/user/device-token
- **Role:** Retailer, Distributor (called from the mobile app on login)
- **Input (Body):**

JSON

```
{ "token": "a_unique_device_token_from_fcm_or_apns" }
```

- **Output (Success):** 200 OK status.

Overview:-

Here is a summary of the modules we have covered:

1. User Authentication & Onboarding 
 2. Admin Panel & Verification 
 3. Product & Inventory Management 
 4. Order & Transaction Management 
 5. AI-Powered Demand Forecasting 
 6. Advertisement Management 
 7. Content & Notifications 
-

Final Step: The Complete Project Document

According to our initial plan, the final step is to combine all this information into a single, comprehensive project document. This document will serve as the complete technical and functional blueprint for the Medistocks platform.

I will structure the final document as follows:

- 1. Introduction: Project aim, scope, and objectives.
- 2. User Roles & Permissions: Defining the capabilities of Admins, Retailers, and Distributors.
- 3. Detailed Module Specifications: Combining the in-depth documentation we've created for each of the 7 modules.
- 4. Consolidated Database Schema: A complete overview of all database tables and their relationships.
- 5. API Endpoint Summary: A full list of all API endpoints for the entire application.

Medistocks: Project Requirements Document

- **Version: 1.0**
- **Date: September 27, 2025**
- **Project: Medistocks - A B2B Marketplace for Near-Expiry Pharmaceutical Products**

1. Introduction

1.1. Project Overview

Medistocks is a digital platform designed to connect pharmaceutical distributors and retailers. The primary goal is to create a marketplace for selling near-expiry pharmacy items at discounted prices, thereby reducing

waste and providing cost-effective inventory for retailers. The platform will facilitate product discovery, secure transactions, and communication, and will be enhanced with features like AI-powered demand forecasting and in-app advertising.

1.2. Key Objectives

- To create a centralized marketplace for near-expiry pharmaceutical goods.
- To provide a secure onboarding and verification process for professional users.
- To facilitate efficient product discovery through location-based filtering.
- To enable secure and verifiable transactions between buyers and sellers.
- To offer data-driven insights to help users optimize their stock.
- To create a monetization channel through user subscriptions and advertisements.

2. User Roles & Permissions

The platform is designed for three primary roles:

Role	Description	Key Permissions
Retailer	A licensed pharmacy owner/manager looking to purchase inventory at a discount.	- Register, subscribe, and manage their profile. - View and filter products on the marketplace. - Send purchase requests and chat with sellers. - Manage their own private inventory. - Create and purchase ad campaigns.

Role	Description	Key Permissions
Distributor	A licensed pharmaceutical distributor looking to sell bulk or near-expiry stock.	- Register, subscribe, and manage their profile. - Post products for sale on the marketplace. - Accept/reject purchase requests and chat with buyers. - Generate invoices and complete transactions. - Create and purchase ad campaigns.
Admin	A platform administrator responsible for management and oversight.	- Manage user verification (approve/reject). - Oversee and manage all platform content (ads, videos). - View platform analytics (revenue, user growth). - Provide user support and manage subscriptions.

Export to Sheets

3. Detailed Module Specifications

This section contains the detailed workflow, schema, and API endpoints for each of the 7 modules that constitute the Medistocks platform.

3.1. Module 1: User Authentication & Onboarding 🔑

- Objective: To provide a secure process for new users to register, verify their identity, subscribe, and log in.
- Workflow:
 1. User verifies their mobile number via OTP.
 2. The system checks if the user is new or returning.
 3. New users select a role (Retailer/Distributor), fill in details (PAN, License No.), and upload documents to AWS S3.

4. User selects a subscription package (e.g., Silver, Gold) and completes the payment.
5. The account is created with a pending status, awaiting admin approval.
6. Returning users with an approved status are logged in.

3.2. Module 2: Admin Panel & Verification

- Objective: To provide a central interface for the admin to manage users, content, and view platform analytics.
- Workflow:
 1. Admin logs in using a secure email/password.
 2. The dashboard displays key metrics: pending verifications, total users, revenue, etc.
 3. User Verification: Admin views pending applications, reviews submitted documents, and either Approves (notifying the user of activation) or Rejects (providing a reason).
 4. Management: Admin can also manage ad requests, upload videos, and view user subscription statuses.

3.3. Module 3: Product & Inventory Management

- Objective: To enable users to list products for sale and manage their private stock.
- Workflow:
 1. Public Marketplace: Approved users can post products for sale, providing details like product name, manufacturer, expiry date, quantity, MRP, and discounted price.
 2. Product Discovery: Retailers can view all listed products and use filters (e.g., location radius, product type) to find relevant items.

3. Private Inventory: All users have a "My Inventory" section to track their personal stock for management purposes. This is separate from the marketplace and private to the user. The system can send alerts for items nearing expiry.

3.4. Module 4: Order & Transaction Management

- Objective: To facilitate a structured and verifiable transaction process from request to completion.
- Workflow:
 1. A Retailer sends a Purchase Request for a specific quantity of a product.
 2. The Distributor receives the request and can Accept or Reject it.
 3. If accepted, a private in-app chat is enabled between the two users for that order.
 4. After negotiation, the Distributor generates an invoice, and the system sends a unique Verification Code to the Retailer.
 5. At the time of goods exchange, the Distributor enters the code provided by the Retailer into their app to Complete the transaction.
 6. After completion, both parties can rate each other.

3.5. Module 5: AI-Powered Demand Forecasting

- Objective: To provide users with data-driven insights on product demand.
- Workflow:
 1. A background system analyzes internal sales data, external weather forecasts, and seasonal trends.
 2. It generates insights like, "Monsoon season in Hyderabad is likely to increase demand for anti-cold medication."

3. These insights are displayed on the user's dashboard, personalized to their location, helping them make smarter stocking decisions.

3.6. Module 6: Advertisement Management 📢

- Objective: To allow users to purchase ad space for promotion and to create a revenue stream for the platform.
- Workflow:
 1. A user creates an ad campaign by uploading a banner, writing a headline, setting a duration, and making a payment.
 2. The ad is submitted with a pending_approval status.
 3. An Admin reviews the ad and either Approves it (making it live) or Rejects it (with a reason).
 4. Approved ads are displayed to other users in designated slots within the app, such as the dashboard.

3.7. Module 7: Content & Notifications 🎬

- Objective: To deliver video content from the admin and keep users informed via a centralized notification system.
- Workflow:
 1. Video Content: Admin uploads videos through their panel. Users can watch these videos in a dedicated "Learning Center" section of the app.
 2. Notifications: The system automatically sends in-app and push notifications for critical events (e.g., new purchase request, chat message, account approval). Users can view a history of their alerts in a notification center.

4. Consolidated Database Schema

Below is a complete list of all database tables required for the Medistocks platform, followed by a diagram illustrating their relationships.

4.1. Table Definitions

- users: Stores core information for Retailers and Distributors.
- admins: Securely stores credentials for platform administrators.
- subscriptions: Tracks the subscription status and package for each user.
- payments: A log of all payment transactions for subscriptions and ads.
- products: Contains all product listings available on the public marketplace.
- user_inventory: Stores the private stock information for each user.
- orders: The central table for tracking the lifecycle of each transaction.
- chat_messages: Stores the conversation history for each order.
- ratings: Contains ratings and reviews submitted by users after a transaction.
- videos: Information about educational/promotional videos uploaded by the admin.
- advertisements: Manages all ad campaigns created by users.
- ad_analytics: Tracks performance metrics (impressions, clicks) for ads.
- demand_forecasts: Stores pre-computed demand insights for different locations.
- notifications: A log of all notifications sent to users.

4.2. Schema Relationship Diagram

This diagram shows how the primary tables are interconnected.

Code snippet

erDiagram

users ||--o{ subscriptions : "has"

users ||--o{ payments : "makes"

users ||--o{ products : "sells"

users ||--o{ user_inventory : "owns"

users ||--o{ orders : "as buyer/seller"

users ||--o{ ratings : "gives/receives"

users ||--o{ advertisements : "creates"

users ||--o{ notifications : "receives"

admins ||--o{ videos : "posts"

products ||--o{ orders : "is for"

orders ||--o{ chat_messages : "has"

orders ||--o{ ratings : "is rated on"

subscriptions ||--o{ payments : "is paid via"

advertisements ||--o{ ad_analytics : "has"

5. API Endpoint Summary

Below is a master list of all API endpoints required for the Medistocks application, grouped by functionality.

5.1. Authentication APIs (/api/auth)

- POST /send-otp: Sends OTP to a mobile number.
- POST /verify-otp: Verifies OTP and logs in or starts registration.
- POST /register: Submits user registration details and documents.

5.2. User & Package APIs (/api)

- GET /user/me: Gets the profile of the logged-in user.
- PUT /user/me: Updates the profile of the logged-in user.
- POST /user/device-token: Registers a device token for push notifications.
- GET /packages: Gets the list of available subscription packages.

5.3. Product & Inventory APIs (/api)

- POST /products: Creates a new product listing for the marketplace.
- GET /products: Gets a filtered, paginated list of products.
- GET /products/{productId}: Gets details of a single product.
- POST /inventory: Adds an item to the user's private inventory.
- GET /inventory: Gets items from the user's private inventory.

5.4. Order & Transaction APIs (/api/orders)

- POST /: Creates a new purchase request.
- GET /: Gets a list of orders (as buyer or seller).
- PUT /{orderId}/status: Accepts or rejects a purchase request.
- GET /{orderId}/chat: Gets the chat history for an order.
- POST /{orderId}/chat: Sends a message in the chat.

- POST /{orderId}/initiate-completion: Generates invoice and sends verification code.
- POST /{orderId}/complete: Completes the transaction using the verification code.
- POST /{orderId}/rate: Submits a rating for a completed order.

5.5. Content & Notification APIs (/api)

- GET /videos: Gets a list of all published videos.
- GET /notifications: Gets the user's notifications.
- POST /notifications/mark-read: Marks notifications as read.

5.6. AI & Advertisement APIs (/api)

- GET /dashboard/forecasts: Gets AI-powered demand forecasts for the user's location.
- POST /advertisements: Creates a new ad campaign request.
- GET /advertisements/my-ads: Gets the logged-in user's ad campaigns.
- GET /advertisements/active: Gets a live ad to display in the app.
- POST /advertisements/{adId}/track: Tracks an ad impression or click.

5.7. Admin APIs (/api/admin)

- POST /login: Logs in an administrator.
- GET /dashboard/stats: Gets platform statistics for the admin dashboard.
- GET /users: Gets a filtered list of platform users.
- GET /users/{userId}: Gets the full details of a specific user.
- PUT /users/{userId}/status: Approves or rejects a user's registration.
- GET /ads: Gets a filtered list of ad campaigns.

- PUT /ads/{adId}/status: Approves or rejects an ad campaign.
- POST /videos: Uploads and publishes a new video.